Domain 3: Deployment, Provisioning, and Automation

AWS Certified SysOps Administrator – Associate (SOA-C03)

> ***Exam Weight:*** *~17% of scored content*
> ***Source:*** *AWS Certified SysOps Administrator – Associate Exam Guide*

Overview

Domain 3 covers the skills needed to provision cloud resources, manage infrastructure as code (IaC), implement deployment strategies, and automate operational processes. A CloudOps engineer must be comfortable with the full lifecycle of resource provisioning — from building machine images to deploying multi-account stacks and automating day-to-day operations using AWS-native and third-party tools.

Task 3.1: Provision and Maintain Cloud Resources

Skill 3.1.1 – Create and Manage AMIs and Container Images (EC2 Image Builder)

What is EC2 Image Builder?

EC2 Image Builder is a fully managed AWS service that automates the creation, management, and deployment of customized, secure, and up-to-date server images — both Amazon Machine Images (AMIs) and container images. It eliminates the manual effort of building and patching images by providing a pipeline-based workflow.

> *Source: What is Image Builder? – EC2 Image Builder*

Key Concepts

Concept	Description
Image Recipe	Defines the base image, build components, and test components used to create an AMI
Container Recipe	Similar to an image recipe but targets Docker container images distributed to Amazon ECR
Pipeline	Automates the build, test, and distribution of images on a schedule or on demand
Component	A YAML-based script (using AWSTOE) that installs software, applies configuration, or runs tests
Infrastructure Configuration	Specifies the EC2 instance type, IAM role, VPC, and SNS topic used during the build
Distribution Configuration	Defines target AWS Regions, accounts, and encryption settings for the output image

Image Builder Workflow

1. **Define a recipe** – Select a base image (AWS-managed or custom) and add build/test components.
2. **Configure infrastructure** – Choose the instance type, IAM instance profile, VPC, and logging settings.
3. **Set distribution** – Specify target Regions and accounts; optionally encrypt with a KMS key.
4. **Run the pipeline** – Image Builder launches a temporary EC2 instance, applies components, runs tests, and distributes the image if all tests pass.
5. **Manage lifecycle** – Use lifecycle policies to automatically deprecate or delete outdated AMIs and container images, reducing storage costs.

AMI vs. Container Image Pipelines

- **AMI pipelines** produce Amazon Machine Images that can be used to launch EC2 instances. They support STIG hardening components, cross-account distribution, and launch template association.
- **Container image pipelines** produce Docker images distributed to Amazon Elastic Container Registry (ECR). They follow the same recipe/component model but output OCI-compliant images.

Cross-Account AMI Distribution

Image Builder supports distributing AMIs to other AWS accounts. The target account must grant the Image Builder service role permission to copy the AMI. You can configure this via the console or AWS CLI, and optionally encrypt the shared AMI with a KMS key managed in the target account.

> *Source: Set up cross-account AMI distribution – EC2 Image Builder*

Security Best Practices

- Apply the latest OS patches as part of the build component.

- Enable IMDSv2 on the build instance.
- Run clean-up scripts to remove temporary credentials, SSH keys, and build artifacts before the image is finalized.
- Integrate Amazon Inspector scanning into the pipeline to detect vulnerabilities before distribution.

Skill 3.1.2 – Create and Manage Stacks with AWS CloudFormation and AWS CDK

AWS CloudFormation

AWS CloudFormation is a service that lets you model, provision, and manage AWS resources using declarative templates written in JSON or YAML. A *stack* is a collection of AWS resources that CloudFormation manages as a single unit.

> *Source: What is CloudFormation? – AWS CloudFormation*

Core Template Anatomy

```
AWSTemplateFormatVersion: "2010-09-09"
  Description: "Example stack"
  Parameters:
    InstanceType:
      Type: String
      Default: t3.micro
  Resources:
    MyEC2Instance:
      Type: AWS::EC2::Instance
      Properties:
        InstanceType: !Ref InstanceType
        ImageId: ami-0abcdef1234567890
  Outputs:
    InstanceId:
      Value: !Ref MyEC2Instance
```

Key CloudFormation Features

Feature	Description
Stacks	A set of AWS resources managed together; create, update, or delete as a unit
Change Sets	Preview proposed changes to a stack before applying them
Drift Detection	Identify resources whose actual configuration differs from the template
Nested Stacks	Embed one stack inside another using `AWS::CloudFormation::Stack` for modular templates
Cross-Stack References	Export outputs from one stack and import them into another using `Fn::ImportValue`
Stack Policies	Protect specific resources from being updated or replaced during stack updates
Rollback Triggers	Automatically roll back a stack update if a CloudWatch alarm fires during deployment
CloudFormation Hooks	Invoke custom logic (Lambda or Guard rules) before or after resource provisioning
IaC Generator	Reverse-engineer existing AWS resources into CloudFormation templates

Stack Lifecycle

1. **Create** – CloudFormation provisions all resources defined in the template.
2. **Update** – Submit a modified template or parameter change; use change sets to preview impact.
3. **Delete** – CloudFormation deletes all resources in the stack (unless a deletion policy is set to `Retain` or `Snapshot`).

Best Practices

- Use **change sets** before every update to understand the impact.
- Store templates in version control (e.g., AWS CodeCommit, GitHub).
- Use **parameters** and **pseudo-parameters** (`AWS::Region`, `AWS::AccountId`) for portability.
- Never embed credentials in templates; use AWS Secrets Manager or SSM Parameter Store dynamic references.
- Enable **termination protection** on production stacks.
- Use **cfn-guard** (CloudFormation Guard) to enforce compliance policies as code.

> *Source: CloudFormation best practices – AWS CloudFormation*

AWS Cloud Development Kit (AWS CDK)

The **AWS CDK** is an open-source software development framework that lets you define cloud infrastructure using familiar programming languages — TypeScript, JavaScript, Python, Java, C#/.NET, and Go. The CDK synthesizes your code into CloudFormation templates and deploys them via CloudFormation.

> *Source: What is the AWS CDK? – AWS CDK v2*

CDK Core Concepts

Concept	Description
App	The root of a CDK application; contains one or more stacks
Stack	Maps 1:1 to a CloudFormation stack; the unit of deployment
Construct	The basic building block; represents one or more AWS resources
L1 Construct	Direct CloudFormation resource mapping (e.g., `CfnBucket`)
L2 Construct	Higher-level abstraction with sensible defaults (e.g., `s3.Bucket`)
L3 Construct (Patterns)	Multi-resource patterns for common architectures (e.g., `ecs_patterns.ApplicationLoadBalancedFargateService`)

CDK Workflow

```
# Initialize a new CDK project
  cdk init app --language typescript

  # Synthesize CloudFormation template
  cdk synth

  # Bootstrap the target environment (one-time per account/region)
  cdk bootstrap aws://ACCOUNT_ID/REGION

  # Deploy the stack
  cdk deploy

  # Destroy the stack
  cdk destroy
```

CDK vs. CloudFormation

Aspect	CloudFormation	AWS CDK
Language	JSON / YAML	TypeScript, Python, Java, C#, Go
Abstraction	Low (resource-level)	High (constructs with defaults)
Reuse	Modules, nested stacks	Constructs, libraries, Construct Hub
Testing	cfn-guard, cfn-lint	Unit tests with Jest, pytest
Output	Template is the artifact	Synthesizes to CloudFormation

Bootstrapping

Before deploying CDK apps to an account/Region for the first time, you must run `cdk bootstrap`. This creates a CloudFormation stack (`CDKToolkit`) that provisions an S3 bucket for assets, ECR repository for container images, and IAM roles used during deployment.

> *Source: AWS CDK bootstrapping*

Skill 3.1.3 – Identify and Remediate Deployment Issues

Deployment failures are common in real-world operations. The exam tests your ability to diagnose and fix issues across several categories.

Common CloudFormation Errors

Error Type	Cause	Remediation
`CREATE_FAILED`	Resource provisioning error (e.g., invalid AMI ID, quota exceeded)	Check the Events tab in the console; fix the template parameter or resource config
`ROLLBACK_COMPLETE`	Stack creation failed and was rolled back	Delete the stack and re-create after fixing the template
`UPDATE_ROLLBACK_FAILED`	Stack update failed and rollback also failed	Use `ContinueUpdateRollback` API to skip problematic resources
`DELETE_FAILED`	A resource could not be deleted (e.g., non-empty S3 bucket)	Manually empty the resource, then retry deletion
Circular dependency	Two resources reference each other	Refactor to break the dependency using outputs/imports

Subnet Sizing Issues

When CloudFormation creates VPC resources, subnet CIDR blocks must:

- Fall within the VPC CIDR range.
- Not overlap with other subnets in the same VPC.
- Be large enough to accommodate the required number of IP addresses (AWS reserves 5 IPs per subnet).

If a subnet is too small for an Auto Scaling group or ECS task placement, deployments will fail with capacity errors. Resize the subnet CIDR or add additional subnets.

Permissions Issues

- The IAM role or user executing the CloudFormation stack must have permissions for every resource type in the template.
- Use a **CloudFormation service role** (`--role-arn`) to grant CloudFormation a specific IAM role, separating deployment permissions from user permissions.

- For CDK deployments, the bootstrapped IAM roles (`cdk-deploy-role`, `cdk-cfn-exec-role`) must have the necessary permissions.

Troubleshooting Tips

- Enable **CloudTrail** to audit all CloudFormation API calls.
- Use the **CloudFormation console Events tab** to see the exact resource and error message.
- For EC2 instances launched by CloudFormation, check `/var/log/cfn-init.log` and `/var/log/cloud-init-output.log` for `cfn-init` errors.
- Use `cfn-signal` with a `WaitCondition` to ensure EC2 instances complete their initialization before CloudFormation marks the stack as complete.

> *Source: Troubleshooting CloudFormation – AWS CloudFormation*

Skill 3.1.4 – Provision and Share Resources Across Multiple Regions and Accounts

AWS CloudFormation StackSets

CloudFormation StackSets extend the functionality of stacks by enabling you to create, update, or delete stacks across multiple AWS accounts and Regions with a single operation.

> *Source: Managing stacks across accounts and Regions with StackSets – AWS CloudFormation*

StackSets Permission Models

Model	Description
Self-managed	You manually create IAM roles (`AWSCloudFormationStackSetAdministrationRole` and `AWSCloudFormationStackSetExecutionRole`) in the administrator and target accounts
Service-managed	Uses AWS Organizations integration; AWS manages the IAM roles automatically; supports automatic deployment to new accounts joining the organization

Key StackSets Concepts

- **Administrator account** – The account from which you create and manage the StackSet.
- **Target accounts** – The accounts where stack instances are deployed.
- **Stack instance** – A reference to a stack in a specific account and Region. An instance can exist even if the stack failed to create.
- **Deployment options** – Control concurrency (how many accounts/Regions deploy simultaneously) and failure tolerance (how many failures are allowed before the operation stops).

StackSets Use Cases

- Deploy a baseline security configuration (e.g., AWS Config rules, CloudTrail) to all accounts in an organization.
- Provision shared networking resources (e.g., VPC, Transit Gateway attachments) across Regions.
- Enforce tagging policies or IAM roles organization-wide.

Troubleshooting StackSets

- **OUTDATED status** – The stack instance is out of sync with the StackSet template. Trigger an update operation.
- **INOPERABLE status** – The stack instance is in a state that prevents operations. Check for manual changes to the stack.
- Ensure the execution role in the target account trusts the administrator account's administration role.

> *Source: Troubleshooting CloudFormation StackSets*

AWS Resource Access Manager (AWS RAM)

AWS RAM lets you securely share AWS resources across AWS accounts, within your AWS Organization or OUs, and with specific IAM roles and users — without duplicating resources.

> *Source: What is AWS Resource Access Manager? – AWS RAM*

How AWS RAM Works

1. The **resource owner** creates a *resource share* and specifies the resources to share and the principals (accounts, OUs, or the entire organization) to share with.
2. If sharing outside the organization, the **recipient** receives an invitation and must accept it.
3. Once accepted, the recipient can use the shared resource as if it were in their own account (subject to the managed permissions defined in the share).

Shareable Resource Types (Examples)

- Amazon VPC subnets and prefix lists
- AWS Transit Gateway and attachments
- Amazon Route 53 Resolver rules
- EC2 Image Builder images and components
- AWS License Manager configurations
- Amazon Aurora DB clusters
- AWS Systems Manager documents

Benefits of AWS RAM

- **Reduces operational overhead** – Create a resource once and share it; no need to duplicate across accounts.

- **Consistent security** – A single set of policies governs access to the shared resource.
- **Auditability** – Integration with CloudTrail and CloudWatch provides visibility into shared resource usage.

RAM vs. Resource-Based Policies

While some resources support cross-account access via resource-based policies (e.g., S3 bucket policies), AWS RAM provides a centralized management experience, invitation workflow, and integration with AWS Organizations that resource-based policies alone do not offer.

Skill 3.1.5 – Implement Deployment Strategies and Services

Choosing the right deployment strategy is critical to minimizing downtime and risk when releasing new application versions.

Deployment Strategy Overview

> *Source: Deployment strategies – Introduction to DevOps on AWS*

Strategy	Description	Downtime	Rollback Speed	Cost
In-place (All-at-once)	Deploy to all instances simultaneously	Possible	Slow (redeploy old version)	Low
Rolling	Replace instances in batches	Minimal	Moderate	Low
Rolling with additional batch	Add new instances before removing old ones	None	Moderate	Slightly higher
Immutable	Launch entirely new instances; swap when healthy	None	Fast (terminate new)	Higher
Blue/Green	Run two identical environments; shift traffic	None	Instant (swap DNS/LB)	Higher
Canary	Shift a small percentage of traffic first; expand if healthy	None	Fast	Moderate

AWS CodeDeploy

AWS CodeDeploy is a fully managed deployment service that automates application deployments to EC2 instances, on-premises servers, AWS Lambda functions, and Amazon ECS services.

> *Source: What is CodeDeploy? – AWS CodeDeploy*

CodeDeploy Compute Platforms

Platform	Deployment Types Supported
EC2/On-Premises	In-place, Blue/Green
AWS Lambda	Canary, Linear, All-at-once
Amazon ECS	Blue/Green (via CodeDeploy + ALB)

AppSpec File

The **AppSpec file** (`appspec.yml`) is the configuration file that CodeDeploy uses to manage a deployment. It defines:

- The files to copy and their destination paths (EC2).
- The Lambda function version to deploy.
- Lifecycle event hooks (scripts to run at each deployment phase).

```
# Example AppSpec for EC2
  version: 0.0
  os: linux
  files:
    - source: /
      destination: /var/www/html
  hooks:
    BeforeInstall:
      - location: scripts/stop_server.sh
    AfterInstall:
      - location: scripts/start_server.sh
    ApplicationStart:
      - location: scripts/validate_service.sh
```

Blue/Green Deployments with CodeDeploy

In a blue/green deployment:

1. CodeDeploy provisions a new set of instances (green environment) and deploys the new application version.
2. Traffic is shifted from the original (blue) environment to the green environment via the load balancer.
3. The blue environment is kept for a configurable period before termination, enabling fast rollback.

Canary Deployments

A canary deployment shifts a small percentage of traffic (e.g., 10%) to the new version first. If CloudWatch alarms remain healthy, the remaining traffic is shifted. If an alarm fires, CodeDeploy automatically rolls back.

For Lambda, CodeDeploy supports configurations like `CodeDeployDefault.LambdaCanary10Percent5Minutes` (shift 10% for 5 minutes, then 100%).

> *Source: Canary deployments – Overview of Deployment Options on AWS*

AWS CodePipeline

AWS CodePipeline is a fully managed continuous delivery service that automates the build, test, and deploy phases of your release process. It integrates with CodeCommit, GitHub, CodeBuild, CodeDeploy, CloudFormation, and third-party tools.

A typical pipeline:

1. **Source** – Triggered by a commit to CodeCommit or GitHub.
2. **Build** – CodeBuild compiles code, runs unit tests, and produces artifacts.
3. **Test** – Integration or acceptance tests.
4. **Deploy** – CodeDeploy or CloudFormation deploys to staging, then production.

CodePipeline can also deploy CloudFormation StackSets, enabling multi-account/multi-Region deployments as part of a CI/CD pipeline.

AWS Elastic Beanstalk

AWS Elastic Beanstalk is a PaaS service that handles the deployment, capacity provisioning, load balancing, auto scaling, and health monitoring of web applications. You upload your code and Beanstalk manages the underlying infrastructure.

Beanstalk supports multiple deployment policies:

- **All at once** – Fastest but causes brief downtime.
- **Rolling** – Deploys in batches; reduces capacity during deployment.
- **Rolling with additional batch** – Maintains full capacity throughout.
- **Immutable** – Deploys to a fresh set of instances; safest option.
- **Traffic splitting** – Canary-style deployment for testing new versions.

Skill 3.1.6 – Use and Manage Third-Party Tools to Automate Resource Deployment

Terraform

Terraform by HashiCorp is an open-source IaC tool that uses a declarative configuration language (HCL) to provision and manage infrastructure across multiple cloud providers, including AWS.

> *Source: Infrastructure as code – Introduction to DevOps on AWS*

Terraform vs. CloudFormation

Aspect	Terraform	CloudFormation
Provider support	Multi-cloud (AWS, Azure, GCP, etc.)	AWS only
Language	HCL (HashiCorp Configuration Language)	JSON / YAML
State management	Terraform state file (local or remote)	Managed by AWS
Drift detection	`terraform plan` shows drift	Built-in drift detection
Import existing resources	`terraform import`	IaC Generator / `resource import`
AWS integration	Via AWS Provider for Terraform	Native

Terraform Workflow

```
# Initialize the working directory
  terraform init

  # Preview changes
  terraform plan

  # Apply changes
  terraform apply

  # Destroy resources
  terraform destroy
```

Terraform State

Terraform tracks the state of managed resources in a **state file** (`terraform.tfstate`). For team environments, store state remotely in an S3 bucket with DynamoDB locking:

```
terraform {
    backend "s3" {
      bucket         = "my-terraform-state"
      key            = "prod/terraform.tfstate"
      region         = "us-east-1"
      dynamodb_table = "terraform-lock"
      encrypt        = true
    }
  }
```

AWS Control Tower Account Factory for Terraform (AFT)

AWS provides **Account Factory for Terraform (AFT)** as a GitOps-based solution for automating AWS account provisioning using Terraform pipelines. AFT integrates with AWS Control Tower to enforce guardrails and customizations on newly vended accounts.

> *Source: Overview of AWS Control Tower Account Factory for Terraform (AFT)*

Git and Source Control

Git is the standard version control system for managing IaC templates, application code, and configuration files. AWS provides **AWS CodeCommit** as a fully managed, private Git repository service.

Key practices for IaC with Git:

- Store all CloudFormation templates, CDK code, and Terraform configurations in Git.
- Use **branching strategies** (e.g., GitFlow, trunk-based development) to manage environment promotions.
- Require **pull request reviews** before merging infrastructure changes.
- Use **pre-commit hooks** or CI checks to run `cfn-lint`, `terraform validate`, or `cdk synth` before merging.
- Tag releases to correlate infrastructure versions with application releases.

Task 3.2: Automate the Management of Existing Resources

Skill 3.2.1 – Use AWS Services to Automate Operational Processes (AWS Systems Manager)

AWS Systems Manager is a unified management service that provides visibility and control over AWS infrastructure. It helps you automate operational tasks, manage configurations, apply patches, and run commands across fleets of EC2 instances and on-premises servers — without requiring SSH or RDP access.

> *Source: What is AWS Systems Manager? – AWS Systems Manager*

SSM Agent

The **SSM Agent** is software installed on managed nodes (EC2 instances, on-premises servers, VMs) that enables Systems Manager to communicate with and manage those nodes. The agent must be installed and the node must have an IAM instance profile with the `AmazonSSMManagedInstanceCore` policy attached.

Key Systems Manager Capabilities

Run Command

Execute shell scripts or PowerShell commands across multiple managed nodes simultaneously without SSH. Results are logged to Amazon S3 or CloudWatch Logs.

```
# Example: Run a shell command on all instances with a specific tag
  aws ssm send-command \
    --document-name "AWS-RunShellScript" \
    --targets "Key=tag:Environment,Values=Production" \
    --parameters "commands=['sudo yum update -y']"
```

Patch Manager

Automates the process of patching managed nodes with security-related updates. You define **patch baselines** (rules for which patches to approve) and **maintenance windows** (scheduled times for patching).

- **Patch Baseline** – Defines which patches are approved for installation (by severity, classification, or specific patch IDs).
- **Patch Group** – A tag-based grouping of instances associated with a specific patch baseline.
- **Maintenance Window** – A scheduled time window during which patching (and other tasks) can run.

Session Manager

Provides browser-based and CLI-based interactive shell access to managed nodes without opening inbound ports, maintaining bastion hosts, or managing SSH keys. All sessions are logged to S3 and CloudWatch Logs for auditing.

State Manager

Maintains a defined configuration state on managed nodes. You associate a Systems Manager document (SSM document) with target instances, and State Manager ensures the configuration is applied and re-applied on a schedule.

Use cases:

- Ensure the CloudWatch agent is always installed and running.
- Enforce specific software versions.
- Apply security configurations.

Inventory

Collects metadata about managed nodes, including installed applications, network configurations, Windows updates, and running services. Data is stored in Systems Manager and can be queried or exported to S3 for analysis with Amazon Athena.

Systems Manager Automation

Automation lets you create and run *runbooks* (SSM documents of type `Automation`) that define a series of steps to perform on AWS resources. Runbooks can call AWS APIs, run scripts, invoke Lambda functions, and wait for approvals.

> *Source: AWS Systems Manager Automation*

Predefined Runbooks

AWS provides hundreds of predefined runbooks (e.g., `AWS-RestartEC2Instance`, `AWS-StopEC2Instance`, `AWSSupport-TroubleshootSSH`) that cover common operational tasks.

Custom Runbooks

You can author custom runbooks in YAML or JSON using the visual designer or a text editor. A runbook step can use action types such as:

Action Type	Description
`aws:runCommand`	Run a command on managed nodes
`aws:executeScript`	Run a Python or PowerShell script inline
`aws:invokeLambdaFunction`	Invoke a Lambda function
`aws:waitForAwsResourceProperty`	Poll a resource property until a condition is met
`aws:approve`	Pause execution and wait for manual approval
`aws:changeInstanceState`	Start, stop, or terminate EC2 instances

Automation Execution Modes

- **Simple execution** – Run the runbook once against a target.
- **Rate control** – Run against multiple targets with concurrency and error thresholds.
- **Multi-account and multi-Region** – Run across accounts and Regions using AWS Organizations.

Troubleshooting Automation

Common errors include:

- **IAM PassRole errors** – The execution role must have `iam:PassRole` permission.
- **VPC 400 errors** – The automation instance cannot reach the Systems Manager endpoint; check VPC endpoints or NAT gateway.
- **Timeout** – Increase the `timeoutSeconds` for long-running steps.

> *Source: Troubleshooting Systems Manager Automation*

Change Manager

Change Manager is an enterprise change management framework within Systems Manager. It provides a structured approval workflow for operational changes, integrating with AWS Organizations for cross-account change management.

Key components:

- **Change template** – Defines the runbook, required approvals, and notification settings for a type of change.
- **Change request** – A specific instance of a change, created from a template and submitted for approval.
- **Approver** – An IAM user, role, or SNS topic that must approve the change before it executes.

Parameter Store

AWS Systems Manager Parameter Store provides secure, hierarchical storage for configuration data and secrets. Parameters can be plain text (`String`, `StringList`) or encrypted (`SecureString` using AWS KMS).

```
# Store a parameter
  aws ssm put-parameter \
    --name "/myapp/prod/db-password" \
    --value "MySecretPassword" \
    --type SecureString \
    --key-id alias/aws/ssm

  # Retrieve a parameter
  aws ssm get-parameter \
    --name "/myapp/prod/db-password" \
    --with-decryption
```

CloudFormation templates can reference Parameter Store values using dynamic references:

```
DBPassword: "{{resolve:ssm-secure:/myapp/prod/db-password}}"
```

OpsCenter

OpsCenter aggregates operational work items (OpsItems) from CloudWatch alarms, EventBridge rules, and other AWS services into a single console. Each OpsItem includes contextual information and links to relevant runbooks for remediation.

Skill 3.2.2 – Implement Event-Driven Automation (AWS Lambda, Amazon S3 Event Notifications)

Event-driven automation responds to changes in your AWS environment automatically, without human intervention. AWS Lambda and Amazon S3 Event Notifications are two of the most common building blocks.

AWS Lambda

AWS Lambda is a serverless compute service that runs your code in response to events. You pay only for the compute time consumed — there is no charge when your code is not running.

> *Source: Creating event-driven architectures with Lambda – AWS Lambda*

Lambda Event Sources

Lambda can be triggered by a wide range of AWS services:

Event Source	Trigger Type
Amazon S3	Object created, deleted, restored
Amazon DynamoDB Streams	Record inserts, updates, deletes
Amazon Kinesis	Data stream records
Amazon SQS	Messages in a queue
Amazon SNS	Published messages
Amazon EventBridge	Scheduled rules, custom events
AWS CloudFormation	Custom resources
Amazon API Gateway	HTTP requests
AWS IoT	Device messages

Lambda Execution Model

- **Synchronous invocation** – The caller waits for the function to complete (e.g., API Gateway).
- **Asynchronous invocation** – The caller does not wait; Lambda retries on failure (e.g., S3, SNS).
- **Event source mapping** – Lambda polls a stream or queue and invokes the function in batches (e.g., SQS, Kinesis, DynamoDB Streams).

Lambda Best Practices for Automation

- Keep functions small and focused on a single task.
- Use **environment variables** for configuration; store secrets in Parameter Store or Secrets Manager.
- Set appropriate **timeouts** and **memory** allocations.
- Use **Dead Letter Queues (DLQ)** (SQS or SNS) to capture failed asynchronous invocations.

- Use **Lambda Destinations** to route successful or failed invocations to SQS, SNS, EventBridge, or another Lambda function.
- Avoid Lambda monoliths; use **AWS Step Functions** for complex multi-step workflows.

Lambda with Systems Manager Automation

Lambda functions can be invoked from Systems Manager Automation runbooks using the `aws:invokeLambdaFunction` action. This enables complex logic (e.g., querying external APIs, making decisions based on resource state) within an automation workflow.

Amazon S3 Event Notifications

Amazon S3 Event Notifications allow you to receive notifications when specific events occur in an S3 bucket, such as object creation, deletion, or restoration.

> *Source: Amazon S3 Event Notifications – Amazon S3*

Supported Event Types

Event	Description
`s3:ObjectCreated:*`	Any object creation (PUT, POST, COPY, multipart upload)
`s3:ObjectRemoved:*`	Object deletion (permanent or delete marker)
`s3:ObjectRestore:*`	Glacier restore initiated or completed
`s3:Replication:*`	Replication events (missed threshold, failed, etc.)
`s3:LifecycleExpiration:*`	Lifecycle expiration events

Notification Destinations

S3 can send event notifications to:

- **AWS Lambda** – Invoke a function to process the event.
- **Amazon SQS** – Queue the event for asynchronous processing.
- **Amazon SNS** – Fan out the event to multiple subscribers.
- **Amazon EventBridge** – Route events to any EventBridge target (most flexible option).

Configuring S3 Event Notifications

```
{
    "LambdaFunctionConfigurations": [
      {
        "LambdaFunctionArn": "arn:aws:lambda:us-east-1:123456789012:function:ProcessUpl
  oad",
        "Events": ["s3:ObjectCreated:*"],
        "Filter": {
          "Key": {
            "FilterRules": [
              { "Name": "prefix", "Value": "uploads/" },
              { "Name": "suffix", "Value": ".csv" }
            ]
          }
        }
      }
    ]
  }
```

> *Source: Enabling and configuring event notifications using the Amazon S3 console*

S3 + Lambda Automation Pattern

A common automation pattern:

1. A file is uploaded to an S3 bucket (e.g., a CSV report).
2. S3 sends an event notification to Lambda.
3. Lambda processes the file (e.g., parses data, writes to DynamoDB, sends an SNS notification).
4. Results are stored back in S3 or another data store.

Important: Avoid creating a trigger loop — do not configure a Lambda function to write back to the same S3 bucket that triggers it without using a prefix/suffix filter to differentiate input and output objects.

> *Source: Process Amazon S3 event notifications with Lambda*

Amazon EventBridge

Amazon EventBridge is a serverless event bus that connects AWS services, SaaS applications, and custom applications using events. It is the recommended way to build event-driven architectures at scale.

EventBridge vs. S3 Event Notifications

Feature	S3 Event Notifications	EventBridge
Targets	Lambda, SQS, SNS	20+ targets including Step Functions, API Gateway, CodePipeline
Filtering	Prefix/suffix only	Rich content-based filtering on any event field
Event archive	No	Yes (replay events)
Schema registry	No	Yes
Cross-account	No	Yes

EventBridge Rules for Automation

EventBridge rules match incoming events and route them to targets. Rules can be:

- **Event pattern rules** – Match events based on their content (e.g., EC2 instance state change to `stopped`).
- **Schedule rules** – Trigger targets on a cron or rate schedule (e.g., run a Lambda function every hour).

```
// Example: Trigger Lambda when an EC2 instance is stopped
  {
    "source": ["aws.ec2"],
    "detail-type": ["EC2 Instance State-change Notification"],
    "detail": {
      "state": ["stopped"]
    }
  }
```

Common EventBridge Automation Patterns

- **Auto-remediation** – Detect a non-compliant resource via AWS Config and trigger a Lambda or Systems Manager Automation runbook to fix it.
- **Scheduled maintenance** – Use a cron rule to trigger a Systems Manager Automation runbook for nightly patching or backups.
- **Cross-account event routing** – Send events from member accounts to a central event bus in a management account for centralized processing.
- **CI/CD triggers** – Trigger CodePipeline when a new image is pushed to ECR.

Key Services Summary for Domain 3

Service	Primary Use in Domain 3
EC2 Image Builder	Automate AMI and container image creation, patching, and distribution
AWS CloudFormation	Declarative IaC for provisioning and managing AWS resource stacks
AWS CDK	Programmatic IaC using general-purpose languages; synthesizes to CloudFormation
CloudFormation StackSets	Deploy stacks across multiple accounts and Regions from a single operation
AWS RAM	Share AWS resources (subnets, Transit Gateways, images) across accounts
AWS CodeDeploy	Automate application deployments with blue/green, rolling, and canary strategies
AWS CodePipeline	Orchestrate CI/CD pipelines integrating build, test, and deploy stages
AWS Elastic Beanstalk	PaaS for deploying web applications with managed infrastructure
AWS Systems Manager	Centralized node management, patching, automation runbooks, and configuration
AWS Lambda	Serverless compute for event-driven automation and custom logic
Amazon S3 Event Notifications	Trigger automation workflows on S3 object events
Amazon EventBridge	Event bus for routing and reacting to events across AWS services and accounts
Terraform	Open-source multi-cloud IaC tool; integrates with AWS via the AWS Provider

---

Exam Tips for Domain 3

1. **CloudFormation errors** – Know the difference between `CREATE_FAILED`, `ROLLBACK_COMPLETE`, and `UPDATE_ROLLBACK_FAILED`. The `ContinueUpdateRollback` API is the fix for stuck rollbacks.
2. **StackSets permission models** – Self-managed requires manual IAM role creation; service-managed uses AWS Organizations and is simpler for org-wide deployments.
3. **AWS RAM vs. resource-based policies** – RAM provides centralized management, invitation workflows, and Organizations integration. Use RAM when sharing resources like VPC subnets across accounts.
4. **Deployment strategies** – Blue/green offers the fastest rollback (instant traffic shift). Canary is best for gradual validation. In-place is fastest to deploy but riskiest.
5. **CodeDeploy AppSpec** – Know the lifecycle event hooks for EC2 (`BeforeInstall`, `AfterInstall`, `ApplicationStart`, `ValidateService`) and Lambda (`BeforeAllowTraffic`, `AfterAllowTraffic`).

6. **Systems Manager prerequisites** – SSM Agent must be installed and the instance must have an IAM instance profile with `AmazonSSMManagedInstanceCore`. For private instances, configure VPC endpoints for Systems Manager.
7. **S3 event notification loops** – A Lambda function triggered by S3 must not write back to the same bucket prefix/suffix that triggers it, or it will create an infinite loop.
8. **EventBridge vs. S3 notifications** – Prefer EventBridge when you need rich filtering, cross-account routing, event archiving, or more than three target types.
9. **CDK bootstrapping** – `cdk bootstrap` must be run once per account/Region before the first CDK deployment. It creates the `CDKToolkit` CloudFormation stack.
10. **Terraform state** – Always use a remote backend (S3 + DynamoDB) for team environments to prevent state file conflicts and enable locking.

References

- EC2 Image Builder User Guide
- AWS CloudFormation User Guide
- AWS CDK v2 Developer Guide
- CloudFormation StackSets
- AWS Resource Access Manager User Guide
- AWS CodeDeploy User Guide
- Overview of Deployment Options on AWS (Whitepaper)
- AWS Systems Manager User Guide
- AWS Lambda Developer Guide
- Amazon S3 Event Notifications
- Amazon EventBridge User Guide
- AWS Control Tower Account Factory for Terraform (AFT)
- Introduction to DevOps on AWS (Whitepaper)